

Algorithm Practice Problems

A collection of problems to practice algorithms

Eklavya Raman
er24ms024@iiserkol.ac.in

July 22, 2025

0.1 Exercises

0.1.1 Easy Problems

1. **Majority Element** - Given an array of size n , find the element which appears the most number of times. If no such unique element exists, return -1.
2. **Two Sum** - Given an array of integers and a target sum, find two distinct indices such that the numbers at those indices add up to the target sum.
3. **Reverse a list** - Write a function that takes a list as input and returns the string reversed.

0.1.2 Intermediate problems

1. **Fibonacci Generation** - Write a liner and recursive program to generate the n^{th} fibonacci number.
2. **List sort** - Write an algorithm which takes a list as an input and outputs the sorted array.
3. **Closest Pair** - Given a set of points in a 2D plane, find the closest pair of points.
4. **Longest Common Subsequence** - Given two arrays, find the length of the longest subsequence present in both arrays.

0.1.3 Hard Problems

These problems are harder and may require more advanced techniques or brute force methods. They are not expected to be solved in a short time, both come in the class NP, with knapsack being NP-complete.

1. **Knapsack Problem** - Given a set of items, each with a weight and a value, determine the maximum value that can be obtained by selecting items without exceeding a given weight limit.
2. **Sudoku Solver** - Write a program that solves a given Sudoku puzzle (puzzle given as a 3D array)

0.2 Solutions

```
# Majority Element
def majority_element(arr):
    count = {}
    for num in arr:
        if num in count:
            count[num] += 1
        else:
            count[num] = 1
    max_count = 0
    majority_element = -1
    for key, value in count.items():
        if value > max_count:
            max_count = value
            majority_element = key
        if value == max_count:
            majority_element = -1
    return majority_element
```

```
# Two Sum
def two_sum(arr, target):
    seen = {}
    for i, num in enumerate(arr):
        complement = target - num
        if complement in seen:
            return (seen[complement], i)
```

```

        seen[num] = i
    return None

# Reverse a list
def reverse_list(lst):
    return lst[::-1]

# Fibonacci Generation
# Linear Generator
def fibonacci_linear(n):
    a, b = 0, 1
    for i in range(n):
        a, b = b, a + b
    return a

# Recursive Generator
def fibonacci_recursive(n):
    if n <= 1:
        return n
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)

# List sort
def list_sort(lst):
    n = len(lst)
    for i in range(n):
        for j in range(0, n-i-1):
            if lst[j] > lst[j+1]:
                lst[j], lst[j+1] = lst[j+1], lst[j]
    return lst

# Closest Pair
import math
def closest_pair(points):
    def distance(p1, p2):
        return math.sqrt((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2)

    min_dist = float('inf')
    pair = None
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            dist = distance(points[i], points[j])
            if dist < min_dist:
                min_dist = dist
                pair = (points[i], points[j])
    return pair

# Longest Common Subsequence
def longest_common_subsequence(arr1, arr2):
    n, m = len(arr1), len(arr2)
    max_len = max(n, m)
    longest = []
    for i in range(n):
        subseq = []
        for j in range(m):
            if arr1[i] == arr2[j]:
                subseq.append(arr1[i])
                for k in range(1, max_len - max(i, j)):
                    if arr1[i+k] == arr2[j+k]:
                        subseq.append(arr1[i+k])

```

```
        else:
            break
    if len(subseq) > len(longest):
        longest = subseq
return longest
```

Knapsack Problem

```
def knapsack(weights, values, capacity):
    def is_valid_combo(weights=weights, capacity=capacity):
        curr_weight = 0
        for i in weights:
            if curr_weight + i > capacity:
                return False
        return True
    import itertools
    max_val = 0
    for r in range(1, len(weights) + 1):
        for combo in itertools.combinations(range(len(weights)), r):
            combo_weights = [weights[i] for i in combo]
            combo_values = [values[i] for i in combo]
            if is_valid_combo(combo_weights):
                total_value = sum(combo_values)
                if total_value > max_val:
                    max_val = total_value
    return max_val
```

Sudoku Solver

```
def sudoku_solver(board):
    def is_valid(board, row, col, num):
        for i in range(9):
            if board[row][i] == num or board[i][col] == num:
                return False
        start_row, start_col = 3 * (row // 3), 3 * (col // 3)
        for i in range(start_row, start_row + 3):
            for j in range(start_col, start_col + 3):
                if board[i][j] == num:
                    return False
        return True

    def solve(board):
        for row in range(9):
            for col in range(9):
                if board[row][col] == 0:
                    for num in range(1, 10):
                        if is_valid(board, row, col, num):
                            board[row][col] = num
                            if solve(board):
                                return True
                            board[row][col] = 0
                    return False
        return True

    solve(board)
    return board
```